

1주차 강의노트 : 시스템 메모리 구조 & 함수 호출 원리

I. 시스템 해킹과 포너블

1. 시스템 해킹(system hacking)

시스템은 키보드, 메인보드, 하드디스크, 램 메모리 등의 하드웨어 뿐만 아니라 운영체제, 데이터베이스, 웹 서비스 등의 소프트웨어까지 매우 많은 것을 포괄한다.¹

시스템 해킹은 컴퓨터 시스템이나 네트워크에 접근하거나 조작하는 행위를 말한다. 권한 탈취, 리버싱, 취약점 공격, 포너블, 디버깅, 익스플로잇(exploit, 공격 코드) 등을 포괄한다.

2. 포너블(pwnable)

포너블은 CTF 대회에서 자주 등장하는 문제 유형으로, 시스템이나 프로그램의 취약점을 이용해 권한을 탈취할 수 있는 문제 유형을 말한다. 주로 메모리 취약점을 이용해 프로그램의 흐름을 바꾸는 문제들이 출제된다. 포너블 문제는 실제 시스템에서 발생할 수 있는 취약점을 기반으로 구성되므로, 보안 교육 및 실무에서도 시스템 해킹 기술을 훈련하는 데 사용된다.

+) TMI : 포너블(pwnable)의 pwn은 원래 "own"(이기다, 장악하다)의 오타에서 시작되었다. 즉, pwnable은 "장악할 수 있는"이라는 뜻으로, 취약점이 있어 뚫을 수 있는 프로그램이라는 뜻이 된다. (오타 하나 낸 게 아직까지도 이어지고 있는 셈...)

3. 관계 (ChatGPT 사용)

```
시스템 해킹
├─ 운영체제 취약점 분석 (권한 상승, 커널 익스플로잇)
├─ 네트워크 기반 공격 (패킷 분석, 서비스 공격)
├─ 소프트웨어 취약점 공격
│   └─ 리버스 엔지니어링
│       └─ **포너블(Pwnable)**
│           ├── 메모리 구조 이해
│           ├── 버퍼 오버플로우(BOF)
│           ├── 포맷 스트링 버그
│           └─ 기타 메모리 취약점
└─ 기타 시스템 내부 공격 (디버깅, 후킹, 인젝션 등)
```

¹ 『정보 보안 개론』(양대일 저) 46p 중 발췌

II. 메모리 구조 4영역

프로세스를 실행시키면 운영체제는 프로세스 메모리를 세그먼트(segment)라는 논리적 영역으로 나누어 관리한다. 프로세스 메모리는 코드 세그먼트, 데이터 세그먼트, 스택 세그먼트, 힙 세그먼트 등으로 나뉘어진다.

1. 코드 세그먼트(Text Segment)

코드 세그먼트 또는 텍스트 세그먼트라고 불린다. 시스템이 알아들을 수 있는 실행 명령어(기계어)가 저장되어 있다. 컴파일 된 함수 코드와 제어 흐름 관련 명령어들이 포함된다. 실행과 읽기 권한만 부여되어 있다.

2. 데이터 세그먼트(Data Segment)

프로그램 실행 시 사용되는 데이터가 들어간다. 일반적으로는 전역 변수(global variable)와 정적 변수(static variable)들이 저장된다. 더 자세히 나누자면 데이터 세그먼트(Data Segment), BSS 세그먼트(Block Started by Symbol), 리소스 데이터 세그먼트(Resource Data Segment, Read-only data segment) 등으로 세분화할 수 있다. 그냥 뭉뚱그려 데이터 세그먼트라고 부르는 경우가 많다.

- ① 데이터 세그먼트 : 초기화된 전역 변수나 정적 변수가 저장된다.
- ② BSS 세그먼트 : 초기화되지 않은 전역 변수나 정적 변수가 저장된다.
- ③ 리소스 데이터 세그먼트 : 읽기 전용의 초기화된 데이터가 저장된다. 주로 상수로 사용되는 데이터가 저장된다.

3. 힙 세그먼트(Heap Segment)

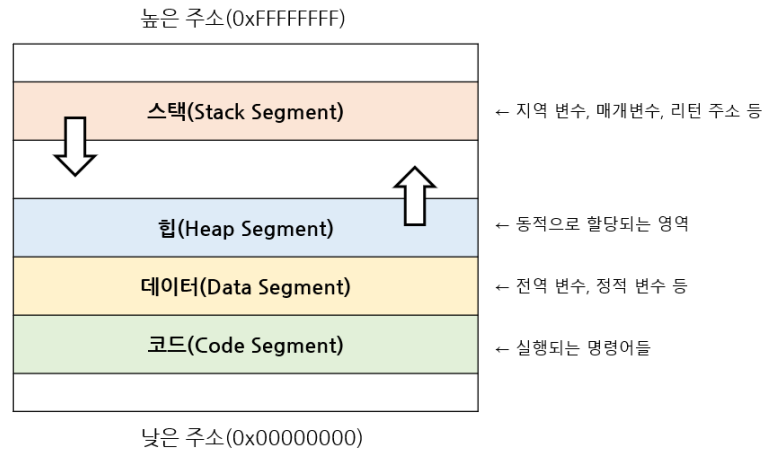
힙 영역은 프로그램이 실행 중에 동적으로 사용되는 메모리 영역이다. 하위 메모리 주소에서 상위 메모리 주소로 확장된다는 특징이 있다.

malloc, calloc, new 등에 의해 실행 중 동적으로 할당되며, 개발자가 직접 free() 또는 delete()로 해제해야 한다. 메모리를 해제하지 않고 반복적인 메모리 할당이 누적되면 시스템 자원이 고갈되어 성능 저하나 비정상 종료로 이어질 수 있다. 이를 메모리 누수(Memory Leak)라고 한다.

4. 스택(stack)

프로그램 로직이 동작하기 위한 인자와 프로세스 상태가 저장되는 영역이다. 함수 호출 시 지역 변수, 매개변수, 반환 주소 등이 저장된다. 상위 메모리 주소에서 하위 메모리 주소로 확장된다는 특징이 있다.

다음 이미지와 같이, 메모리 영역에서 힙은 하위 주소에서 상위 주소로, 스택은 상위 주소에서 하위 주소로 확장된다.



III. 레지스터(80x86 CPU, 80386 레지스터)²

레지스터는 CPU 내부의 연산용 임시 저장소로, CPU 연산과 어셈블리어의 동작에 사용된다. 레지스터는 범용 레지스터, 세그먼트 레지스터, 포인터 레지스터, 인덱스 레지스터, 플래그 레지스터 등으로 구분되며, 각각의 레지스터는 특정 연산이나 메모리 접근에 특화되어 있다.

1. 범용 레지스터(General-Purpose register)

데이터 저장, 연산, 주소 계산 등 다목적으로 사용되는 레지스터이다. 논리 연산과 수리 연산에 사용되는 피연산자, 주소를 계산하는데 사용되는 피연산자, 메모리 포인터 등이 저장된다. 아래는 각 레지스터들의 관례적 역할이다.

레지스터	비트	설명
EAX (Accumulator)	32	연산 결과, 함수 리턴 값 저장.
EBX (Base)	32	특정 주소 저장. 주소 계산을 위해 사용.
ECX (Counter)	32	루프 반복 횟수나 시프트 비트 수 기억.
EDX (Data)	32	일반 데이터 저장. 곱셈/나눗셈 시 보조.

² 표는 『정보 보안 개론』(양대일 저) 227-228p, 『x64dbg 디버거를 활용한 리버싱과 시스템 해킹의 원리』(김민수 저) 38-40p, ChatGPT의 설명을 참고하여 작성함.

2. 세그먼트 레지스터(Segment register)

코드 세그먼트, 데이터 세그먼트, 스택 세그먼트를 가리키는 주소(base 주소)가 들어있는 레지스터이다.

레지스터	비트	설명
CS (Code Segment)	16	실행할 코드가 있는 세그먼트.
DS (Data Segment)	16	일반 데이터가 있는 세그먼트.
SS (Stack Segment)	16	스택 영역의 베이스 주소.
ES, FS, GS	16	여분의 레지스터.

+) TMI :

32비트 아키텍처에서 세그먼트 레지스터가 16비트인 이유는 초창기 x86이 16비트 체계였고, 이때의 세그먼트 레지스터를 그대로 가져왔기 때문이다. 초창기에는 코드, 데이터, 스택이 서로 다른 세그먼트에 불연속적으로 자리잡고 있었다. 그렇기에 세그먼트 레지스터가 가리키는 각 메모리의 시작 주소에 오프셋을 더해 실제 메모리 주소를 계산했다. (실제 메모리 주소 = 세그먼트 * 16 + 오프셋)

그러나 현재에는 Flat Memory Model을 사용하며, OS가 메모리 영역(Code, Data, Heap, Stack)을 연속된 가상 주소 공간 위에 배치한다. 그렇기 때문에 보통 모든 세그먼트 레지스터를 동일한 값으로 세팅한다. 즉, 세그먼트는 더 이상 의미가 없으며 사실상 무시 가능하다.

현재에 와서는 세그먼트 레지스터는 주소 계산을 위해 사용되지 않는다. 그러나 GDT(Global Descriptor Table)나 LDT(Local Descriptor Table)를 통한 권한 레벨 구분, TLS(Thread Local Storage) 접근, 리얼 모드(Real Mode) 등에서는 여전히 사용된다.³ 그러므로 지금은 깊게 다루지 않지만 공부해두면 좋다.

3. 포인터 레지스터(Pointer register)

포인터 레지스터는 원래 범용 레지스터에 속하지만, 스택 프레임, 스택 최상단, 명령어 포인터 등 주소를 가리키는 특별한 용도로 자주 쓰인다. 그래서 일부 교재에서는 이해를 돕기 위해 범용 레지스터와 구분하여 포인터 레지스터로 설명한다.

³ Copilot, ChatGPT 설명 참고 (확인 필요)

레지스터	비트	설명
EBP (Base Pointer)	32	베이스 포인터. 함수 호출 시 스택 프레임의 기준점.
ESP (Stack Pointer)	32	스택 포인터. 스택 최상단(top)을 가리킴.
EIP (Instruction Pointer)	32	명령 포인터. 다음에 실행할 명령어의 주소.

4. 인덱스 레지스터(Index register)

배열, 문자열 처리 등에서 인덱스로 사용하는 레지스터이다.

레지스터	비트	설명
EDI (Destination Index)	32	목적지 인덱스. 목적지 주소의 값 저장.
ESI (Source Index)	32	출발지 인덱스. 출발지 주소의 값.

5. 플래그 레지스터(Flag register)

연산 결과 및 시스템 상태와 관련된 여러 가지 플래그 값들이 저장된다. 80386부터 EFLAGS(32비트)가 사용되며, 상태 플래그, 컨트롤 플래그, 시스템 플래그로 구성되어 있다. 주요 하위 플래그들은 아래와 같다.

분류	레지스터	설명
상태 플래그	CF (Carry Flag)	연산 수행에서 carry 혹은 borrow 발생.
	PF (Parity Flag)	최하위 비트가 0인 경우.
	AF (Adjust Flag)	산술 연산에서 BCD 숫자 캐리 발생.
	ZF (Zero Flag)	연산 결과가 0인 경우.
	SF (Sign Flag)	연산 결과가 음수인 경우.
	OF (Overflow Flag)	산술 연산 결과가 표현 가능 범위 초과.
제어 플래그	IF (Interrupt Enable Flag)	인터럽트가 사용가능한 경우
	DF (Direction Flag)	설정 시 문자열 연산에서 역순으로 읽음.
시스템 플래그	VM (Virtual 8086 Mode)	Virtual 8086 모드 활성화 여부.
	RF (Resume Flag)	디버깅 시 예외 재발 방지.
	AC (Alignment Check)	메모리 정렬 검사

[인텔 80x86 CPU 80386 플래그 레지스터(EFLAGS)]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
										I D	V I P	V I F	A C	V M	R F		N T	IOPL		O F	D F	I F	T F	S F	Z F		A F		P F		C F

IV. 함수 호출 시 스택 변화

프로그램을 실행하면 프로세스가 메모리에 적재되고 앞서 살펴본 레지스터들이 동작하게 된다. 프로그램을 실행 중 함수가 호출되면, 메모리 스택에는 매개변수, 지역변수, 함수 종료 시 되돌아갈 리턴 주소 등이 저장된다. 이렇게 호출된 함수의 정보 틀을 스택 프레임(stack frame)이라 한다.

아래의 코드를 예시로 사용해보자. (주석은 실행 순서)

```
#include <stdio.h>

int sum(int x, int y) {           // ④ sum 함수 진입 → 스택에 sum 프레임 생성
    return x + y;                // ⑤ sum 실행 → x+y 계산 후 리턴 → sum 프레임 제거
}

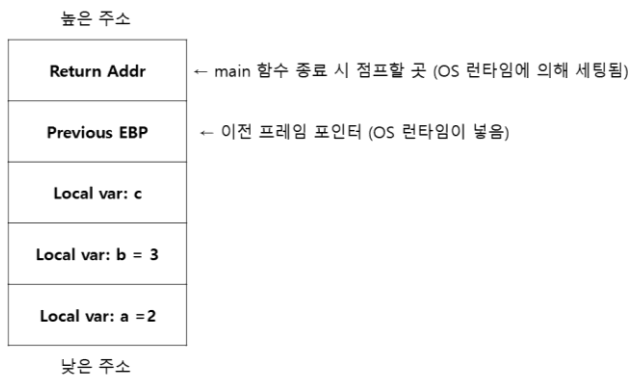
int main() {                     // ① main 함수 진입 → 스택에 main 프레임 생성
    int a = 2, b = 3, c;          // ② 지역 변수 저장 (a, b, c)
    c = sum(a, b);                // ③ sum 함수 호출 → ④~⑤ 수행 → ⑥ 리턴값을 c에 저장
}                                 // ⑦ main 종료 → main 프레임 제거 → 프로그램 종료
```

1. main() 실행

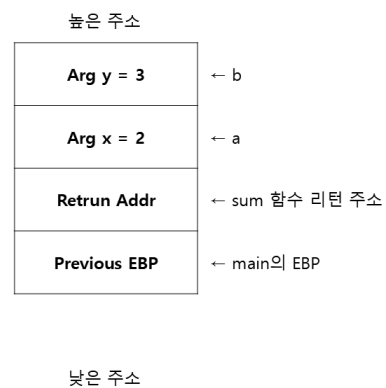
프로그램이 시작되면서 main() 함수에 진입하고, 스택에 main 프레임이 생성된다. 먼저 return address가 스택에 저장되고(①), 그 위로 이전 EBP(Base Pointer)와 지역 변수들을 선언 순서대로 저장된다(②). 이후 c = sum(a, b)에서 sum()함수가 호출된다(③).

EVI\$ION YB 강의세션 - 시스템 해킹(pwnable)

main() 함수 진입 시 (초기 스택)



sum(a, b) 함수 호출 시



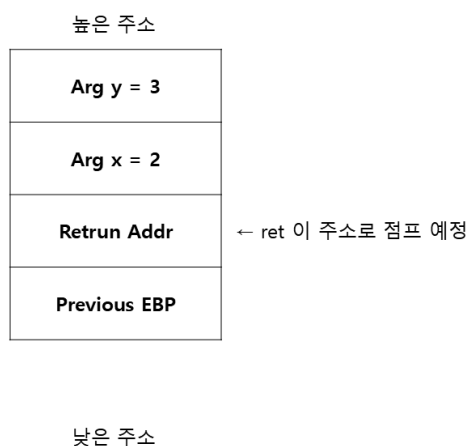
2. sum() 실행

sum() 함수에 진입하여, sum 함수용 스택 프레임을 새로 생성된다(④). 함수가 실행되며 $x + y$ 연산을 수행하고, 결과는 eax 레지스터에 저장된다. 이후 ret 명령어로 리턴하며, sum 프레임은 제거되고 main()으로 복귀한다(⑤).

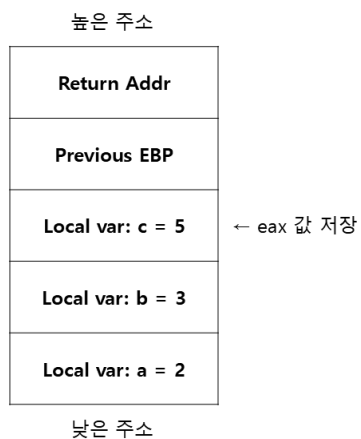
3. main() 복귀

sum()의 결과가 저장된 eax 값이 main()함수의 지역 변수 c에 저장된다(⑥). 이후 main() 함수가 종료되면서, main 프레임도 제거되고 프로그램이 종료된다(⑦).

sum(a, b) 실행 후 리턴 직전 (ret 직전)



sum 리턴 → main에서 c = 5 저장



4. 프로로그와 에필로그

- 1) 프로로그 : 함수 시작 시 스택 프레임 생성. 이전 함수의 EBP 값을 저장하고, 현재 ESP 값을 EBP에 복사함(현재 함수의 프레임 기준점 설정). 로컬 변수 공간 확보
- 2) 에필로그 : 함수 종료 시 프레임 제거. 스택 포인터를 다시 복원하고, 리턴 주소로 점프해서 이전 함수로 복귀

[참고자료]

『정보 보안 개론 4판』, 양대일, 한빛아카데미, 2023.

『x64dbg 디버거를 활용한 리버싱과 시스템 해킹의 원리』, 김민수, 인피니티북스, 2020.

(달고나)와우해커 BOF 기초문서

위키피디아, 「x86 메모리 분할」

(https://ko.wikipedia.org/wiki/X86_%EB%A9%94%EB%AA%A8%EB%A6%AC_%EB%B6%84%ED%95%A0)

위키피디아, 「플랫 메모리 모델」 (https://en.wikipedia.org/wiki/Flat_memory_model)

OSDev Wiki, 「Segmentation」 (<https://wiki.osdev.org/Segmentation>)

AI 보조 (Copilot, ChatGPT) - 내용 정리 및 설명 보조에 활용